

05 — Data Model & API (BetConstructAI)

Reference for the mobile team. Backend is **Supabase** (Postgres + auto-generated PostgREST REST API) plus **Vercel serverless functions** in `api/*.js`.

- **Supabase project ref:** `smddtvaewmmpuyvtuscb`
- **Base URL:** `https://smddtvaewmmpuyvtuscb.supabase.co`
- **REST base:** `https://smddtvaewmmpuyvtuscb.supabase.co/rest/v1/`
- **Anon (publishable) key** — public, safe to ship in the app: `sb_publishable_oPGh3c7PEFreALu0kY6w0A_WovnaNpL`
- **service_role key / other secrets** — server-only, `<REDACTED>`. Never in the mobile app.

The web frontend talks to Supabase directly from the browser with `supabase-js`
(`db.from('table').select/insert/update/upsert/delete`, `db.rpc(...)`, `db.storage.from('uploads')...`) using the anon key. A handful of privileged operations go through Vercel functions that hold the `service_role` key.

Secret scan of copied files: the SQL migrations and CSVs in `mock_data/` were grepped for `service_role` /JWT (`eyJ...`) secrets — **none found**. The only key present is the public anon `sb_publishable...` key. Plaintext **staff login passwords** originally appeared in `add_staff.sql`, `add_sales_team.sql`, and `sales_team.csv`; because they grant access to the live system they have been **replaced with** `<REDACTED>` in the copied files (the `pass` column and row structure are preserved so the data shape is still clear).

Important nuances the mobile team must know

1. **Two "schema generations" exist.** `database.sql` is the original strict-RLS design (auth.users-based profiles, `deals.id` as `uuid`, role-gated policies). The prototype then diverged via `add_*.sql` / `fix_*.sql` / `open_rls_prototype.sql`: RLS was opened to `using(true)`, `deals` was **dropped and recreated with a text PK** (`fix_deals.sql`), a `staff` login table replaced auth for the CRM, and CRM tables (`accounts`, `contacts`, `agreements`, `tasks`, `threads`, `documents`, ...) were added. **The live app uses the prototype generation.** Where the two disagree (esp. `deals`), the prototype wins — both are documented below.
2. **RLS is effectively OPEN.** Historically every prototype table's policy is `for all using (true)` with `check (true)` (or open select/insert). So the anon key can read/write almost everything. Treat the whole DB as readable/writable from the client. (See §C.)
3. **Password columns are now REVOKE'd from anon.** After a recent security change, the anon/authenticated roles can **no longer read** `staff.pass` or `applications.portal_pass` (column-level `REVOKE`). Any `select('*')` from the client silently omits those columns. **Login is verified server-side only** by `POST /api/login` (holds `service_role`), which returns a profile **without** the password. Mobile must authenticate through `/api/login`, never by reading `staff.pass`.
4. `comm_messages`, `email_accounts`, `contacts` **have no CREATE in the repo**. They are referenced by the UI and the serverless functions but their `create table` statements are **not in any .sql file** in this project (created ad-hoc in the Supabase dashboard). Their columns below are **reconstructed from usage** and marked `INFERRED`.
5. `/api/advisor` **on disk targets Google Gemini, not Anthropic**. The brief says advisor "now targets Anthropic Claude Haiku 4.5 / needs `ANTHROPIC_API_KEY`", but **the code currently in this repo (`api/advisor.js` and root `advisor.js`) calls Google Gemini with `GEMINI_API_KEY`**. Documented as-is below with a note. If an Anthropic migration lands, only the upstream provider call changes — the `/api/advisor` request/response contract (`{prompt, context, lang}` → `{text}`) is designed to stay the same.

A) DOMAIN MODEL

Legend: **PK** primary key · **FK** foreign key · `nullable` unless marked **NOT NULL** · example values drawn from seed rows / mock data / realistic mocks.

Enum types (from `database.sql`):

- `user_role`: visitor | sales | lawyer | moderator | marketing | finance | admin
- `app_kind`: game | payment | platform | tool | license | affiliate | marketing
- `app_status`: pending | approved | partner | rejected | new (new = "New offer" badge, added by `add_status_new.sql`)
- `deal_stage`: new | contact | demo | negotiation | purchase (original deals only; prototype deals uses free-text stage)

applications — partner/affiliate/vendor onboarding submissions (KYB queue)

Source: `database.sql` , extended by `add_portal.sql` . Public inserts (marketplace forms), moderated in admin.

field	type	null	notes / enum / FK	example
id	uuid	NOT NULL	PK, default <code>gen_random_uuid()</code>	9f1c2ab4-...
kind	app_kind	NOT NULL	enum	game
company	text	NOT NULL		LuckyStar Games Ltd
contact_name	text	yes		Maria Petrosyan
email	text	NOT NULL		sales@luckystar.io
phone	text	yes		+37491234567
site	text	yes		https://luckystar.io
country	text	yes		Malta
reg_no	text	yes	company reg number	C-88213
licenses	text	yes		MGA/B2C/394/2017
product_name	text	yes		LuckyStar Slots API
category	text	yes		Slots
description	text	yes		2000+ HTML5 slot games...
value_prop	text	yes		Highest RTP catalog in CIS
features	text	yes		Tournaments, jackpots, free spins
demo_url	text	yes		https://demo.luckystar.io
regions	text	yes		CIS, LatAm, Europe
forbidden_markets	text	yes		US, France
languages	text	yes		EN, RU, HY
currencies	text	yes		EUR, USD, AMD
crypto	boolean	yes	default false	true
compliance	text	yes		GLI-19 certified
model	text	yes	pricing model	RevShare
price	text	yes	free text	15% GGR
platform_share	text	yes		5%
ai_tags	jsonb	yes	AI-generated tags	["slots","crypto","cis"]
status	app_status	NOT NULL	default pending	approved
submitted_by	uuid	yes	FK profiles.id	null
reviewed_by	uuid	yes	FK profiles.id	null
reviewed_at	timestampz	yes		2025-07-15T10:22:00Z

field	type	null	notes / enum / FK	example
created_at	timestampz	yes	default now()	2025-07-14T09:00:00Z
portal_pass	text	yes	add_portal.sql . REVOKE'd from anon — not readable by client	<REVOKED>
portal_active	boolean	yes	default false (add_portal.sql)	true
portal_reset_requested	timestampz/text	yes	INFERRED (used in admin.html) — partner requested password reset	2025-08-01T12:00:00Z
signer_name	text	yes	INFERRED (selected in admin.html)	Maria Petrosyan

vendors — approved applications published to the Marketplace catalog

Source: database.sql + add_clicks.sql , add_impressions.sql , add_vendor_desc.sql .

field	type	null	notes / FK	example
id	uuid	NOT NULL	PK	c3a0...
application_id	uuid	yes	FK applications.id	9f1c2ab4-...
name	text	NOT NULL		LuckyStar Games
kind	app_kind	yes	enum	game
category	text	yes		Slots
badge	app_status	yes	default approved ; `approved	partner
match_score	int	yes	0-100 (UI sets 65 for new , 80 otherwise)	80
regions	text	yes		CIS, Europe
active	boolean	yes	default true	true
created_at	timestampz	yes	default now()	2025-07-15T11:00:00Z
clicks	int	yes	default 0 (add_clicks.sql) — "Request demo" clicks	12
impressions	int	yes	default 0 (add_impressions.sql) — marketplace views	340
description	text	yes	add_vendor_desc.sql — shown in catalog + fed to AI advisor	2000+ HTML5 slots

listings — extra media/detail rows under a vendor (original schema; little UI use)

Source: database.sql .

field	type	null	FK	example
id	uuid	NOT NULL	PK	...
vendor_id	uuid	yes	FK vendors.id (ON DELETE CASCADE)	c3a0...
title	text	yes		Premium Slots Pack
description	text	yes		Top 100 games
media	jsonb	yes		{"images":["..."]}
price	text	yes		€5000/mo
created_at	timestampz	yes	default now()	...

staff — CRM login accounts (sales/support/admin team). Replaces auth for admin+staff cabinets

Source: add_staff.sql , extended by add_shift_ranks.sql ; seeded by add_sales_team.sql / sales_team.csv .

field	type	null	notes	example
id	uuid	NOT NULL	PK	a1b2...
name	text	yes		Gayane Arakelyan
role	text	yes	free text: `Sales`	Support
login	text	yes	UNIQUE	gayane.arakelyan
pass	text	yes	REVOKE'd from anon — not readable by client; checked only in /api/login	<REVOKED> (demo: 0hbVrpoi)
active	boolean	yes	default true	true
created_at	timestamptz	yes	default now()	...
shifts_total	int	yes	default 0 (add_shift_ranks.sql)	15
shifts_weekend	int	yes	default 0 (add_shift_ranks.sql)	6
email	text	yes	INFERRED (admin select/insert) — used by email-inbound/gmail matching	gayane@betconstruct.com
phone	text	yes	INFERRED (admin select)	+37491000000
whatsapp	text	yes	INFERRED	+37491000000
telegram	text	yes	INFERRED	@gayane
gmail_connected	boolean	yes	INFERRED — set true by gmail-auth-callback	false

profiles — original auth.users extension (strict schema; largely bypassed by staff in prototype)

Source: database.sql .

field	type	null	FK / enum	example
id	uuid	NOT NULL	PK, FK auth.users.id (ON DELETE CASCADE)	<auth uid>
full_name	text	yes		Armen Kazaryan
contact	text	yes	email or phone	armen@betconstruct.com
role	user_role	NOT NULL	default visitor	admin
online	boolean	yes	default false	true
last_seen	timestamptz	yes		2025-09-09T08:00:00Z
created_at	timestamptz	yes	default now()	...

leads — "Request demo" / cart-interest leads from marketplace visitors

Source: add_leads.sql (+ owner from add_deals.sql).

field	type	null	notes	example
id	uuid	NOT NULL	PK	...
vendor	text	yes	vendor name or Корзина интересов (cart)	LuckyStar Games
vendor_id	uuid	yes	(loose ref to vendors.id)	c3a0...
product	text	yes		LuckyStar Slots API
visitor_name	text	yes		Ivan
contact	text	yes	email/phone	ivan@casino.com

field	type	null	notes	example
message	text	yes		Interested in slots demo
source	text	yes	default demo_request ; also cart	demo_request
status	text	yes	default new ; `new	in_progress`
created_at	timestampz	yes	default now()	...
owner	text	yes	add_deals.sql — staff name who claimed it	Gayane Arakelyan

deals — CRM deals (PROTOTYPE version: text PK)

Source: fix_deals.sql / add_deals.sql (recreated with text id). **This is the live version.**

field	type	null	notes	example
id	text	NOT NULL	PK — app codes like BC-1042	BC-1042
client	text	yes		LuckyStar Games
resp	text	yes	responsible (Sales) name	Gayane Arakelyan
ssales	text	yes	Service Sales name	Narek Avetisyan
pm	text	yes	Project Manager name	David Khachatryan
am	text	yes	Account Manager name	Gor Mnatsakanyan
stage	text	yes	default lead (free text pipeline stage)	demo
status	text	yes	default active	active
agr	jsonb	yes	default {"signed":false,"paid":false}	{"signed":true,"paid":false}
flow	jsonb	yes	default [false×7] — 7-step checklist	[true,true,false,false,false,false,false]
steps	jsonb	yes	default [] — activity log	[{"name":"Demo sent","at":"..."}]
rr_done	boolean	yes	default false — requirements review done	false
kick_done	boolean	yes	default false — kickoff done	false
created_at	timestampz	yes	default now()	...

Original deals (database.sql, superseded): id uuid PK, code text unique, client text, responsible uuid FK profiles, stage deal_stage default 'new', amount numeric default 0, created_at. Also had companion tables deal_steps (id bigint, deal_id FK, name, by_name, at) and payouts (id uuid, sales_id FK profiles, amount, status default 'pending', created_at). These are dropped/unused after fix_deals.sql.

accounts — registry of "whose client/partner" + imported records

Source: add_accounts.sql. RLS open.

field	type	null	notes	example
id	uuid	NOT NULL	PK	...
name	text	yes	company/client	LuckyStar Games
kind	text	yes	default client ; `client	partner
contact_name	text	yes		Maria Petrosyan
email	text	yes		maria@luckystar.io

field	type	null	notes	example
phone	text	yes		+37491234567
owner	text	yes	staff name owner	Gayane Arakelyan
owner_role	text	yes		Sales
stage	text	yes		negotiation
source	text	yes	`Zoho	DocuSign
notes	text	yes		Met at SiGMA 2025
created_at	timestamptz	yes	default now()	...

contacts — people under an account (INFERRED — no CREATE in repo)

Reconstructed from admin.html insert + serverless matching (account_id, email, phone, whatsapp, telegram).

field	type	null	notes	example
id	uuid	NOT NULL	PK (INFERRED)	...
account_id	uuid	yes	FK accounts.id	...
name	text	yes		Maria Petrosyan
position	text	yes		Head of Sales
email	text	yes	used to match inbound email → account	maria@luckystar.io
phone	text	yes		+37491234567
whatsapp	text	yes		+37491234567
telegram	text	yes		@maria_ls
created_at	timestamptz	yes	INFERRED	...

comm_messages — unified communication log (email/whatsapp/telegram/call/note) (INFERRED — no CREATE in repo)

Reconstructed from api/comm-ingest.js , api/email-inbound.js , api/gmail-*.js , and admin.html .

field	type	null	notes	example
id	uuid/bigint	NOT NULL	PK (INFERRED)	...
account_id	uuid	yes	FK accounts.id (null if unmatched)	...
staff_name	text	yes	manager name	Gayane Arakelyan
channel	text	yes	`email	whatsapp
direction	text	yes	`in	out
subject	text	yes		Re: Slots demo
body	text	yes	message text / snippet	Thanks, can we schedule a call?
external_id	text	yes	provider/gmail message id (dedup key)	18f2a1bc9d...
created_at	timestamptz	yes	INFERRED	...

email_accounts — connected Gmail mailboxes per staff (INFERRED — no CREATE in repo)

Reconstructed from api/gmail-auth-callback.js , gmail-send.js , gmail-sync.js . **Holds refresh_token — read only via service_role; never expose to the app.**

field	type	null	notes	example
id	uuid	NOT NULL	PK (INFERRED)	...

field	type	null	notes	example
staff_id	uuid	yes	FK staff.id	a1b2...
staff_name	text	yes		Gayane Arakelyan
email	text	yes	mailbox address	gayane@betconstruct.com
provider	text	yes		gmail
refresh_token	text	yes	SECRET <REDACTED> — service_role only	<REDACTED>
last_sync	timestampz	yes	set by gmail-sync	2025-09-09T08:00:00Z

agreements — contract lifecycle (DocuSign replacement)

Source: add_agreements.sql .

field	type	null	notes	example
id	uuid	NOT NULL	PK	...
deal_id	text	yes	(loose ref to deals.id)	BC-1042
partner	text	yes		LuckyStar Games
title	text	yes		Integration Agreement
template	text	yes		standard_b2b
body	text	yes	contract text	This agreement...
doc_url	text	yes	uploaded file URL (Storage)	https://.../uploads/agr.pdf
amount	numeric	yes	setup fee	5000
currency	text	yes	default EUR	EUR
signer_name	text	yes		Maria Petrosyan
signer_email	text	yes		maria@luckystar.io
status	text	yes	default draft ; `draft	sent
sent_at	timestampz	yes		2025-08-01T10:00:00Z
signed_at	timestampz	yes		2025-08-02T14:00:00Z
paid_at	timestampz	yes		null
created_at	timestampz	yes	default now()	...

tasks — internal tasks & resource requests (Jira replacement)

Source: add_tasks.sql .

field	type	null	notes	example
id	uuid	NOT NULL	PK	...
title	text	yes		Integrate slots API
description	text	yes		Set up sandbox creds
type	text	yes	default dept_task ; `resource_request	dept_task
dept	text	yes	`PM	Service Sales
assignee	text	yes	staff name	David Khachatryan
requester	text	yes	staff name	Gayane Arakelyan
deal_id	text	yes		BC-1042
priority	text	yes	default P3 ; `P1	P2
status	text	yes	default open ; `open	in_progress

field	type	null	notes	example
due_date	date	yes		2025-09-20
created_at	timestampz	yes	default now()	...

documents — file storage registry (Supabase Storage bucket uploads)

Source: add_docs.sql . Bucket uploads is **public**, open read/insert/delete.

field	type	null	notes	example
id	uuid	NOT NULL	PK	...
title	text	yes		Signed Agreement
category	text	yes	default Общее ; e.g. Договоры	Договоры
file_url	text	yes	public Storage URL	https://.../uploads/x.pdf
file_type	text	yes	`image	video
file_name	text	yes		agreement.pdf
size_bytes	bigint	yes		284512
uploaded_by	text	yes		Система
vendor_id	uuid	yes	(loose ref)	c3a0...
created_at	timestampz	yes	default now()	...

threads — chat threads (visitor / internal / kickoff) (Hoory/Pandayo replacement)

Source: add_chat.sql .

field	type	null	notes	example
id	uuid	NOT NULL	PK	...
kind	text	yes	default visitor ; `visitor	internal
title	text	yes		Гость 4F2A
deal_id	text	yes		BC-1042
partner	text	yes		LuckyStar Games
guest_key	text	yes	visitor anon binding	anon-9f1c...
created_at	timestampz	yes	default now()	...
last_at	timestampz	yes	default now() — bumped on new message	2025-09-09T08:05:00Z

thread_messages — messages within a thread (Supabase Realtime-enabled)

Source: add_chat.sql .

field	type	null	notes / FK	example
id	uuid	NOT NULL	PK	...
thread_id	uuid	yes	FK threads.id (ON DELETE CASCADE)	...
sender	text	yes	display name	Менеджер
role	text	yes	default visitor ; `staff	visitor
text	text	yes		Hello! How can I help?
file_url	text	yes	attachment (Storage)	https://.../uploads/img.jpg
file_type	text	yes	`image	video
created_at	timestampz	yes	default now()	...

events — anonymous visitor analytics events

Source: database.sql (read opened by add_shifts.sql).

field	type	null	notes	example
id	bigint	NOT NULL	PK identity	4021
visitor_anon	text	NOT NULL	anon id	anon-9f1c...
session_id	text	yes		sess-abc123
type	text	NOT NULL	`page	click
target	text	yes	screen/button/link	vendor_card_luckystar
meta	jsonb	yes		{"x":120}
created_at	timestamptz	yes	default now()	...

visitors — anonymous device tracking

Source: database.sql.

field	type	null	notes / FK	example
id	uuid	NOT NULL	PK	...
anon_id	text	NOT NULL	UNIQUE — from localStorage	anon-9f1c...
fingerprint	text	yes	FingerprintJS	fp_abc123
device	jsonb	yes	ua/screen/tz/lang	{"ua":"...", "lang":"ru-RU", "tz":"Asia/Yerevan"}
geo	text	yes		Yerevan, AM
user_id	uuid	yes	FK profiles.id	null
first_seen	timestamptz	yes	default now()	...
last_seen	timestamptz	yes	default now()	...

shifts — staff presence (online/offline) sessions

Source: add_shifts.sql.

field	type	null	notes	example
id	uuid	NOT NULL	PK	...
staff	text	NOT NULL	staff name	Gayane Arakelyan
role	text	yes		Sales
online_at	timestamptz	yes	default now()	2025-09-09T08:00:00Z
offline_at	timestamptz	yes	set on logout	2025-09-09T17:00:00Z

Other original tables (database.sql , minimal/no prototype UI use)

- chats — id uuid PK, visitor_anon text, visitor_name text, rep_id uuid FK profiles, online boolean, created_at. Superseded by threads .
- messages — id bigint PK, chat_id uuid FK chats (CASCADE), sender text (visitor|staff) NOT NULL, body text NOT NULL, created_at; file_url / file_type added by add_docs.sql . Superseded by thread_messages .
- banners — id uuid PK, title, subtitle, active boolean default true, position int default 0, created_at. Public read, admin write.

- `content_flags` — key text **PK**, enabled boolean default true. Seeded: `quick, live, promo, sections` (all true). Public read.
- `kb_docs` — id uuid PK, title, url, content, created_at (RAG knowledge base).
- `kb_chunks` — id bigint PK, doc_id uuid FK kb_docs (CASCADE), content, `embedding vector(768)` (pgvector; Gemini text-embedding-004 dim).

Postgres functions / RPC:

- `bump_click(vid uuid)` → void — `update vendors set clicks=clicks+1`. Granted to anon. Called `db.rpc('bump_click',{vid})`.
- `bump_impression(vid uuid)` → void — `increments vendors.impressions`. Granted to anon. Called `db.rpc('bump_impression',{vid})`.
- `my_role()`, `is_staff()`, `is_admin()`, `handle_new_user()` — RLS helpers (original auth model).

B) DATA OPERATIONS

B.1 Supabase REST (PostgREST) — client, anon key

All client DB calls go to `https://smddtvaewmmpuyvtscb.supabase.co/rest/v1/<table>` via supabase-js. Under the hood:

- Headers (every request):** `apikey: sb_publishable_oPGh3c7PEFreALu0kY6w0A_WovnaNpL` `Authorization: Bearer sb_publishable_oPGh3c7PEFreALu0kY6w0A_WovnaNpL` `Content-Type: application/json` (writes) `Prefer: return=representation` (to get inserted rows back), `resolution=merge-duplicates` (upsert)
- Methods:** `select` → `GET`, `insert` → `POST`, `update` → `PATCH`, `upsert` → `POST` (+merge), `delete` → `DELETE`.
- Filters/ordering** are query params: `?col=eq.value`, `?select=a,b`, `?order=created_at.desc`, `?limit=50`. `supabase-js .eq()/ .order()/ .limit()` map to these.
- Auth requirement:** anon key only (no user JWT / no Supabase Auth session in the prototype). RLS is open (`using(true)`), so calls succeed for anyone with the anon key — **except** the REVOKE'd columns `staff.pass` and `applications.portal_pass`, which are omitted from responses.
- Success:** `2xx` with a JSON array of rows (or `[]`). **Error:** PostgREST error object `{ "code", "message", "details", "hint" }` with `4xx`.

Enumerated operations (table → ops → where used)

Table	Operations (from <code>db.from(...)</code>)
applications	<code>insert(payload)</code> ; <code>select(...large col list...)</code> ; <code>update({status, reviewed_at,...})</code> , <code>update({portal_reset_re</code>
vendors	<code>select('*')</code> ; <code>insert({application_id,name,category,description,badge,active,regions,match_score})</code> ; <code>delete()</code>
leads	<code>insert(lead)</code> , <code>insert({vendor:'Корзина интересов',...,source:'cart'})</code> ; <code>select('*')</code> ; <code>update({owner,status:</code>
deals	<code>upsert({id,client,resp,ssales,pm,am,stage,status,agr,flow,steps,rr_done,kick_done})</code> ; <code>select('*')</code>
accounts	<code>select('*')</code> , <code>select('id')</code> ; <code>insert(row)</code> ; <code>update(r)</code> , <code>update({owner})</code> ; <code>delete()</code>
contacts	<code>select('*')</code> ; <code>insert({account_id,name,position,email,phone,whatsapp,telegram})</code> ; <code>delete()</code>
comm_messages	<code>select('*')</code> ; <code>insert({account_id,channel,direction,staff_name,subject,body})</code>
agreements	<code>select('*')</code> ; <code>insert(row)</code> ; <code>update({status,sent_at/signed_at/paid_at})</code> , <code>update({doc_url})</code> ; <code>delete()</code>
tasks	<code>select('*')</code> ; <code>insert(row/rows)</code> ; <code>update({status})</code> , <code>update({assignee})</code> ; <code>delete()</code>
documents	<code>select('*')</code> ; <code>insert(row)</code> ; <code>delete()</code>

Table	Operations (from db.from(...))
threads	select('*/*/'id') ; insert({kind,title,guest_key,deal_id,partner}) ; update({last_at})
thread_messages	select('*/*/'created_at') ; insert({thread_id,sender,role,text,file_url,file_type})
staff	select('id,name,role,login,active,shifts_total,shifts_weekend,email,phone,whatsapp,telegram,gmail_connect') ; insert({name,role,login,pass,active,email}) ; update({role}) , update({pass}) , update({active}) , update({facebook_id}) ; delete()
shifts	select('*/*') ; insert({staff,role,online_at}) ; update({offline_at})
events	insert({visitor_anon,session_id,type,target,meta}) ; select('visitor_anon,session_id,type,target,meta,created_at')
visitors	upsert({anon_id,device,...})
profiles	select('role')
RPC bump_click	db.rpc('bump_click',{vid})
RPC bump_impression	db.rpc('bump_impression',{vid})
Storage uploads	storage.from('uploads').upload(...) , .getPublicUrl(...)

Example — insert an application (POST /rest/v1/applications):

```
POST /rest/v1/applications
apikey: sb_publishable_oPGh3c7PEFreALu0kY6w0A_WovnaNpL
Authorization: Bearer sb_publishable_oPGh3c7PEFreALu0kY6w0A_WovnaNpL
Content-Type: application/json
Prefer: return=representation

{ "kind":"game","company":"LuckyStar Games Ltd","email":"sales@luckystar.io",
  "product_name":"LuckyStar Slots API","category":"Slots","regions":"CIS, Europe","crypto":true }
```

Success 201 :

```
[{ "id":"9f1c2ab4-1e00-4c2a-b6d1-3f0a...", "kind":"game", "company":"LuckyStar Games Ltd",
  "email":"sales@luckystar.io", "status":"pending", "created_at":"2025-09-09T08:00:00Z" }]
```

Error 400 :

```
{ "code":"23502","message":"null value in column \"company\" violates not-null constraint",
  "details":null,"hint":null }
```

Example — list active vendors (GET):

```
GET /rest/v1/vendors?select=*&active=eq.true&order=match_score.desc&limit=50
```

```
[{ "id":"c3a0...", "name":"LuckyStar Games", "kind":"game", "category":"Slots",
  "badge":"partner", "match_score":80, "regions":"CIS, Europe", "active":true,
  "clicks":12, "impressions":340, "description":"2000+ HTML5 slots" }]
```

B.2 Serverless functions (Vercel) — POST /api/*

CORS: all allow Access-Control-Allow-Origin: *, POST, OPTIONS , Content-Type . Non-POST → 405
{"error":"POST only"} (except gmail-auth-* which are GET).

POST /api/login — server-side credential check (holds service_role <REDACTED>)

The only correct way to authenticate admin/staff. Reads `staff` (incl. `pass`) via `service_role`, returns profile **without** the password.

- **Env:** `SB_SERVICE_KEY` `<REDACTED>`, `SB_URL`, `ADMIN_LOGINS` (`"email:pass1,email2:pass2"`).
- **Auth:** none required to call (public endpoint); it validates the supplied login/pass itself.
- **Request body:** `{ "kind": "admin" | "staff" | "auto"(default/omit) , "login": string, "pass": string }` `login` is lowercased+trimmed server-side.
- **Diagnostics:** `{ "kind": "_diag" } → { adminLoginsCount, emailsLower:[...], hasSvc:bool }.`

Logic:

- `kind:'admin'` → checks `ADMIN_LOGINS` env, else a `staff` row whose `role` matches `/(админ|admin)$/i` and `active!=false` . → `{ok, profile}` or `{ok:false}` .
- `kind:'staff'` → `staff?login=eq.<login>&active=eq.true` , matches `pass` . → `{ok, profile}` or `{ok:false}` .
- **omit / auto** (unified `/login`) → tries admin, then staff; returns `dest` .

Requests / responses:

```
POST /api/login { "login":"gayane.arakelyan", "pass":"0hbVrpoi" }
```

Success (auto, staff match):

```
{ "ok": true, "dest": "staff",  
  "profile": { "id":"a1b2...", "name":"Gayane Arakelyan", "role":"Sales",  
               "login":"gayane.arakelyan", "shifts_total":0, "shifts_weekend":0 } }
```

Success (auto, admin match):

```
{ "ok": true, "dest": "admin",  
  "profile": { "name":"Администратор", "email":"admin@betconstruct.com", "role":"admin" } }
```

`kind:'admin' / kind:'staff'` return the same profile **without** `dest` : `{ "ok":true, "profile":{...} }`. Failure: `{ "ok": false }` (HTTP 200). Errors: 400 `{ "ok":false, "error":"login and pass required" }`, 500 `{ "error":"SB_SERVICE_KEY not set" }`, 405 `{ "error":"POST only" }`. Note: profile never contains `pass`. The frontend (`login.html`) sends `{login,pass}` (auto) and routes on `d.dest` .

POST /api/advisor — AI advisor proxy (currently Google Gemini; brief expects Anthropic Claude Haiku 4.5)

Proxies a RAG-style prompt to an LLM with platform knowledge as context. Key stays server-side.

- **Env (as coded):** `GEMINI_API_KEY` `<REDACTED>` . (Brief's target state: `ANTHROPIC_API_KEY` `<REDACTED>` — *not present in current code.*)
- **Auth:** none.
- **Request body:** `{ "prompt": string (required), "context": string (platform KB), "lang": "ru"|"en"|"hy" (default "ru") }`
- **Upstream (current code):** `api/advisor.js` tries models `gemini-3.5-flash`, `gemini-2.5-flash-lite`, `gemini-pro-latest`, `gemini-flash-lite-latest` in order (root `advisor.js` uses single `gemini-2.5-flash`), `temperature:0.4` , with a strict system prompt ("answer ONLY from platform data").

Request:

```
POST /api/advisor  
{ "prompt":"Нужен слот-провайдер для СНГ с криптой, бюджет 5000 EUR",  
  "context":"<BetConstruct + Marketplace + Partners knowledge base>", "lang":"ru" }
```

Success 200 :

```
{ "text": "Рекомендация: LuckyStar Games (Slots, CIS, крипто)...\nПредварительная рекомендация, не юрид  
  "model": "gemini-2.5-flash-lite" }
```

Errors: 400 `{ "error":"prompt required" }`, 500 `{ "error":"GEMINI_API_KEY not set" }` (would be `ANTHROPIC_API_KEY not set` after migration). If all models fail: 200 `{ "text":"Не удалось получить`

ответ.", "errors": [...], "availableModels": "..."}).

Contract note for mobile: request {prompt, context, lang} and response {text} are stable across the Gemini→Anthropic switch; only model string and upstream errors differ.

POST /api/notify — email notifications on new application (via Resend)

- **Env:** RESEND_API_KEY <REDACTED> (required); NOTIFY_ADMIN (default corpfundme@gmail.com); NOTIFY_FROM (default BetConstructAI <onboarding@resend.dev>).
- **Auth:** none.
- **Request body:** the application payload (any of company, kind, product_name, category, description, value_prop, features, model, price, regions, languages, currencies, compliance, licenses, country, reg_no, contact_name, email, phone, site).
- **Behavior:** sends (1) admin notice to NOTIFY_ADMIN; (2) if email present, a confirmation to the registrant.

Request: { "company": "LuckyStar Games

Ltd", "kind": "game", "email": "sales@luckystar.io", "product_name": "LuckyStar Slots API" } Success 200: { "ok": true } Errors: 500 {"error": "RESEND_API_KEY not set"}, 502 {"ok": false, "errors": ["resend 4xx: ..."] }.

POST /api/comm-ingest — universal inbound communication receiver (writes comm_messages)

Any channel posts a normalized message; endpoint resolves the client by contact and writes a row. Uses anon key (RLS open on comm_messages).

- **Env:** SB_URL, SB_ANON_KEY (defaults to public key), optional COMM_INGEST_SECRET (shared secret).
- **Auth:** optional shared secret in body (secret); if COMM_INGEST_SECRET set and mismatched → 401 {"error": "bad secret"}.
- **Request body:** { channel, direction, from, to, subject, body, external_id, staff_name, account_id?, secret? }
 - channel: email | whatsapp | telegram | call | chat | note (default note)
 - direction: in | out | note (default note)
 - resolves account_id (if not given) by matching from / to against contacts.email|phone|whatsapp|telegram.

Success 200: { "ok": true, "matched": true, "id": "<comm_messages id>" } Errors: 401 {"error": "bad secret"}, 502 {"ok": false, "error": "<supabase error>" }.

POST /api/email-inbound — inbound email webhook (Mailgun/SendGrid/Cloudflare → comm_messages)

Accepts JSON or form-urlencoded from email providers. Determines direction by matching from / to against staff.email; matches client via contacts.email. Uses anon key.

- **Env:** SB_URL, SB_ANON_KEY.
- **Request fields (any provider aliases):** from|sender|From, to|recipient|To, subject|Subject, text|body-plain|stripped-text|body|plain, Message-Id|message-id|message_id.
- **Logic:** From = staff → direction: 'out', client=To; To = staff → direction: 'in', client=From; else direction: 'in', no staff.

Success 200: { "ok": true, "direction": "in", "matched": true, "staff": "Gayane Arakelyan" } Errors: 400 {"error": "no addresses"}, 502 {"ok": false, "error": "..."}.

Gmail integration (per-staff OAuth) — hold service_role <REDACTED> + Google secrets <REDACTED>

Common env: GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT (default https://betconstructai.com/api/gmail-auth-callback), SB_SERVICE_KEY.

- GET /api/gmail-auth-start?staff=<staff_id> — redirects (302) to Google OAuth consent. Scopes: gmail.readonly gmail.send openid email, access_type=offline&prompt=consent, state=<staff_id>. Errors: 500 GOOGLE_CLIENT_ID not set.

- GET /api/gmail-auth-callback?code&state — exchanges code→tokens, fetches mailbox email, upserts email_accounts (deletes prior rows for that staff, inserts {staff_id,staff_name,email,provider:'gmail',refresh_token}), sets staff.gmail_connected=true. Returns an HTML success page. Errors: 400 no code / no refresh_token, 500 Gmail env not set.
- POST /api/gmail-send — { staff_id, to, subject, body, account_id? }. Refreshes access token from stored refresh_token, sends via Gmail API from the staff's mailbox, then logs an outgoing comm_messages row (via anon key). Success 200 {"ok":true,"id":"<gmail msg id>"} .Errors: 400 {"error":"staff_id, to, body required"} / "Gmail не подключён у сотрудника", 502 {"error":"token refresh failed"}| "gmail send failed",detail}, 500 {"error":"Gmail env not set"}.
- GET /api/gmail-sync (Vercel Cron or manual) — for each email_accounts row: refresh token, list messages newer_than:2d -in:chats (maxResults 25), dedupe by external_id, match client via contacts.email, insert comm_messages, update email_accounts.last_sync. Success 200 {"ok":true,"accounts":N,"imported":M}. Errors: 500 {"error":"Gmail env not set"}|...}.

Note: api/env-check.js was **deleted** (present as D in a sibling checkout) — no diagnostic env endpoint ships.

C) Security posture (must-read for mobile)

- **Supabase project ref:** smddtvaewmmpuyvtuscb. Public anon key sb_publishable_oPGh3c7PEFreALu0kY6w0A_WovnaNpL is safe to embed.
- **RLS was/is OPEN** (using(true)). The strict role-based policies in database.sql / fix_policies.sql were superseded by open_rls_prototype.sql and by every add_*.sql prototype table shipping for all using(true) with check(true) (or open select/insert). Net effect: with the anon key the client can read and write essentially all rows in all prototype tables. This is a **prototype-grade** posture — the SQL comments explicitly warn to restore strict policies + Supabase Auth before production.
- **Passwords are now column-REVOKE'd from anon/authenticated.** staff.pass and applications.portal_pass are no longer selectable by the client, even though row RLS is open. So a select('*') on staff returns every column *except* pass. **Auth must go through** POST /api/login (service_role, server-side), which returns a profile without the password and, for the unified login, a dest of 'admin' or 'staff'.
- **Secrets that must stay server-side (never in the app):** SB_SERVICE_KEY (service_role) <REDACTED>, GEMINI_API_KEY / ANTHROPIC_API_KEY <REDACTED>, RESEND_API_KEY <REDACTED>, GOOGLE_CLIENT_SECRET <REDACTED>, email_accounts.refresh_token <REDACTED>, COMM_INGEST_SECRET <REDACTED>.
- **Demo credentials in mock data:** add_staff.sql, add_sales_team.sql, and sales_team.csv contain **plaintext demo staff passwords** (e.g. gayane.arakelyan / 0hbVrpoi). These are expected demo/seed data and are retained in mock_data/ unchanged — treat them as non-production sample credentials.

Appendix — MOCKED / no-real-backend items

- comm_messages, contacts, email_accounts: **no CREATE TABLE in repo** — columns above are INFERRED from usage; the live shape lives only in the Supabase dashboard.
- staff.email/phone/whatsapp/telegram/gmail_connected and applications.portal_reset_requested/signer_name: referenced by UI/serverless but **no migration adds them** — added ad-hoc in dashboard (INFERRED).
- /api/advisor: upstream is **Gemini in current code** (brief expects Anthropic) — provider is effectively MOCKED /in-flux relative to the brief.
- listings, chats, messages, deal_steps, payouts, banners, kb_docs, kb_chunks: original-schema tables with little/no prototype UI wiring.

Mock data files

Copied unchanged into mobile_export/mock_data/ (24 files): database.sql, add_accounts.sql, add_agreements.sql, add_chat.sql, add_clicks.sql, add_deals.sql, add_docs.sql, add_impressions.sql, add_leads.sql, add_portal.sql, add_sales_team.sql, add_shift_ranks.sql, add_shifts.sql, add_staff.sql, add_status_new.sql, add_tasks.sql, add_vendor_desc.sql, delete_demo_deals.sql,

delete_demo_staff.sql , fix_deals.sql , fix_policies.sql , open_rls_prototype.sql , sales_team.csv ,
admin_i18n.csv .